

MINI-PROJECT SHORT TECHNICAL REPORT

Course: Cross-Platform Mobile App Development (VKU)
Mini-Project Title: Mini-Project 1.2 — Capacitor: Field Survey
Team / Student Name: Nguyen Van Khang
Submission Date: [TODO: DD/MM/YYYY]

1. GENERAL INFORMATION & DELIVERABLE LINKS

- Team Members:**
 - Nguyen Van Khang — Student ID: 23IT121 — Role: Sole developer (Capacitor integration, native plugins, Android build, frontend, API) — Contribution: 100%
-  **Live Demo URL:** [TODO: APK download link] · Web build of the same code: <https://university-field-survey-pwa.vercel.app>
-  **GitHub Repository:** <https://github.com/khgngx/University-Field-Survey-PWA-Capacitor.git>
-  **Video Demo (Optional):** None — the app is demonstrated live.

2. FEATURE IMPLEMENTATION CHECKLIST

#	Required Feature	Status	Implementation Details & Acceptance Level
1	Wrap the web app as a native Android app	 Complete	Capacitor 8 project (<code>appId: vn.edu.vku.fieldsurvey</code> , <code>webDir: dist</code>). The Vite production build is copied into the APK and served from the WebView at <code>https://localhost</code> . <code>minSdk 26</code> , <code>target/compile SDK 36</code> .
2	One-command APK build	 Complete	<code>npm run build:apk</code> runs web build → <code>npx cap sync android</code> → <code>gradlew assembleDebug</code> and checks that the APK exists. The script selects JDK 21 and the Android SDK itself and stops with a clear message if either is missing. Output: debug APK, about 39 MB.
3	Camera plugin — survey photo	 Complete	<code>@capacitor/camera</code> opens the native camera in step 4. The photo is downscaled to at most 1280 px and re-encoded as JPEG (quality 0.7) before it is stored in IndexedDB. Cancelling the camera is handled as "no photo", not an error.
4	Barcode Scanner plugin — room QR	 Complete	<code>@capacitor/barcode-scanner</code> scans a room QR (<code>building\ floor\ room</code>) and fills step 1. A code that is not a room QR is rejected with a message. Fallback without the camera: upload a saved QR image, decoded offline.
5	Geolocation plugin — GPS stamp	 Complete	<code>@capacitor/geolocation</code> adds latitude, longitude and accuracy on submit. Hard 5–6 s limit: if there is no fix or permission is denied, the survey is saved without a location instead of blocking.

6	Network plugin — connectivity	 Complete	@capacitor/network drives the offline banner and triggers the sync queue when the connection returns.
7	Local Notifications plugin	 Complete	@capacitor/local-notifications shows "Đã đồng bộ thành công N khảo sát" after a sync. Permission is requested once, from the Submit tap; the app never prompts in the background.
8	Android permissions	 Complete	Manifest declares INTERNET, ACCESS_NETWORK_STATE, CAMERA, ACCESS_FINE/COARSE_LOCATION, POST_NOTIFICATIONS (Android 13+). Camera and GPS are marked required="false" so devices without them can still install the app.
9	Offline data collection inside the APK	 Complete	The whole UI is bundled in the APK, so it starts with no network. Surveys are stored in IndexedDB inside the WebView and queued for sync with retry and exponential backoff.
10	Single codebase for web and native	 Complete	The same source builds the PWA and the APK. Every Capacitor call lives in src/services/; in the browser the Camera plugin falls back to @ionic/pwa-elements.
11	Sync with the cloud API from the APK	 Complete	The APK is built with an absolute VITE_API_URL; the API allows the WebView origin https://localhost through CORS.
12	Sync while the app is closed	 Not available	The Background Sync API does not exist in the Android WebView. Surveys sync the next time the app is opened or regains a connection.
13	Tested on a device	 Complete	The APK was installed and run on an Android device. [TODO: device model and Android version]
14	iOS build	 Not done	No ios/ platform was added; it needs Xcode and @capacitor/ios. Out of scope for this submission.

Known limitation: on the QR generator page the "download PNG" link may not save a file inside the Android WebView; the share button or a screenshot is the workaround.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

Stack: React 19 + Vite · Capacitor 8 (Android) · Dexie (IndexedDB) · Tailwind CSS 4 · Vercel Serverless + Supabase.

Directory structure (Capacitor-related parts)

capacitor.config.json	appId, appName, webDir	
android/	Native Android project generated by Capacitor (Gradle, manifest, icons)	
scripts/build-apk.sh	Web build -> cap sync -> assembleDebug, with JDK/SDK checks	
src/services/	The only code that imports Capacitor plugins	
camera.service.js	takePhoto()	-> compressed JPEG Blob or null
scanner.service.js	scanRoomQr()	-> { building, floor, room } or null
geo.service.js	getCurrentPosition()	-> { lat, lng, accuracy } or null
network.service.js	getNetworkStatus(), onNetworkChange()	
notify.service.js	requestNotificationPermission(), notifySynced()	
src/db/ · src/sync/	Storage and sync engine (no Capacitor imports)	
src/pages/ · components/	UI, calls services only through plain async functions	

Layering

```

React pages / hooks
  | plain async functions returning plain data
  ▼
src/services/*.service.js → Capacitor plugin → native Android API
  |                                     |
  |                                     └─ in a browser: web implementation / pwa-elements
  ▼
IndexedDB (Dexie) → sync queue → POST {VITE_API_URL}/surveys → Supabase

```

Pages never import a plugin directly. Each service returns ordinary values (a `Blob`, a location object, `null`), so the UI is identical on web and Android and the rest of the code can be unit-tested without a device (120 tests).

State management flow

IndexedDB is the single source of truth; there is no global state library. The wizard writes a draft (autosaved after 500 ms), Submit moves it to `PENDING_SYNC`, and the sync engine moves it through `SYNCING` to `SYNCED` or `FAILED`. The queue and dashboard pages subscribe with a Dexie live query and re-render on every change. Native plugins only feed this flow: the camera supplies the photo, the scanner the room, GPS the coordinates, and the Network plugin the trigger to sync.

Exception handling strategy

- **User cancels a native screen** (camera, scanner): detected and returned as `null`, so nothing is shown as an error.
- **Permission denied or hardware missing:** GPS and notifications are optional and never block a submit; camera and scanner errors are shown next to the button, and the QR image upload remains as a path that needs no camera.
- **GPS never hangs the form:** the plugin timeout is raced against a hard timer, because the permission dialog is not counted by the plugin's own timeout.
- **Sync errors are classified:** network errors, timeouts, 5xx, 408 and 429 are retried with

backoff $\min(2^n \times 5 \text{ s}, 5 \text{ min})$ up to 5 times; other 4xx fail at once with the server's message and wait for a manual retry.

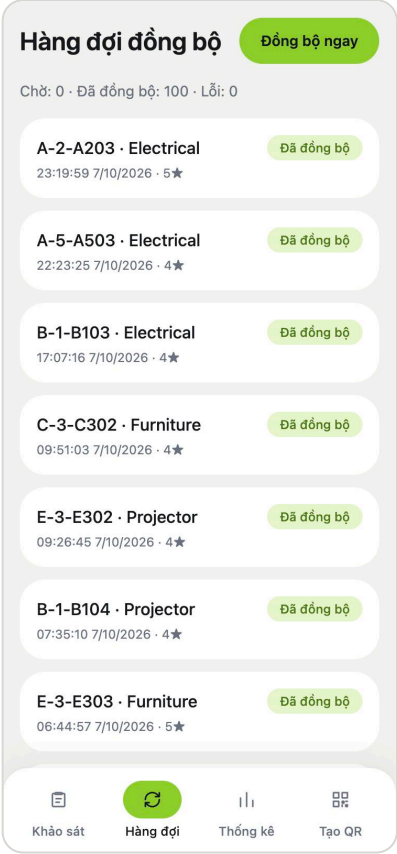
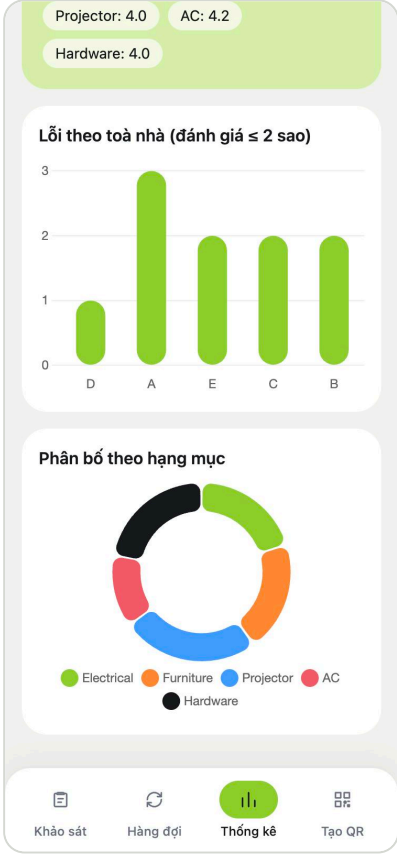
- **Interrupted sync:** a record left in SYNCING when the app was killed is re-queued at the next start.

4. EMPIRICAL EVIDENCE & SCREENSHOTS

[TODO: add 2–3 photos or screenshots from the Android device – suggested: the native QR scanner or camera screen, the permission dialog, and the "synced N surveys" notification. These cannot be captured from a computer.]

The screens below are the same web bundle that is packaged into the APK, captured in a 375 × 812 mobile viewport in a browser (not on the device), with 100 sample surveys.

1. Survey wizard — step 1	2. Queue — waiting to sync
"Quét QR phòng" opens the native Barcode Scanner on Android; "Tải ảnh QR lên" is the camera-free fallback.	Surveys saved on the device with status "Chờ đồng bộ" while the server has not been reached.

3. Queue — after sync	4. Offline dashboard
	
After the Network plugin reports a connection, the queue is sent one survey at a time; all 100 end as "Đã đồng bộ".	Statistics computed from data stored on the device only, so they work with no network.

5. TECHNICAL CHALLENGES & RESOLUTIONS

Challenge 1 — The API could not be reached from inside the APK. On the web the app calls a relative `/api`, which works because the page and the API share one origin. Inside the Capacitor WebView the page is served from `https://localhost`, so the same relative path pointed at `https://localhost/api`, which does not exist, and every sync failed. Calling the real server instead is a cross-origin request, which the browser engine blocks unless the server allows it. *Resolution:* the API base URL became a build-time setting (`VITE_API_URL`) that must be an absolute URL for the APK build, and the API's CORS allow-list always includes the WebView origin `https://localhost` (other origins come from an environment variable). The upload is also idempotent — the server upserts by the survey's UUID — so a retry after a half-finished request cannot create a duplicate.

Challenge 2 — No background sync in the WebView, and a build that failed on the installed JDK. (a) The PWA uses the Service Worker Background Sync API to send queued surveys after the tab is closed, but that API does not exist in the Android WebView, so relying on it would leave surveys unsent on the APK. *Resolution:* background sync is treated as an optional extra. The main path is a set of in-app triggers that work everywhere: the Capacitor Network plugin's `networkStatusChange` event, the `online` event, a 30-second poll, a pass at app start, and a

manual "Đồng bộ ngay" button. A lock prevents two runs from sending the same survey at once.

(b) Android Gradle Plugin 8.13 / Gradle 8.14 reject JDK 25, the system default, and the build stopped with an unclear error. *Resolution:* `scripts/build-apk.sh` looks for JDK 21 explicitly (`JAVA_HOME` , Homebrew, then `java_home -v 21`), verifies the Android SDK path, and exits with a one-line instruction when either is missing.